Python OOP Basics

Md. Ehsanul Haque
Senior Software Quality Assurance Engineer
Vivasoft Ltd.

Agenda

This session will provide a basic understanding of object oriented programming in Python.

- **Python OOP Basics**
  - Class
  - Object
  - Constructor(__init__)
  - Self
  - Class variable
  - Instance variable
  - Inheritance

What is __init__?

- __init__ is a **constructor** method.
- It **runs automatically** whenever you create an object.
- Its job: **initialize (set up) the object's data**.

Note: Think of it as the **"setup"** or **"starting point"** of an object.

```
class Student:
    def __init__(self, name, age):
        # this code runs automatically when a new Student is
created
        self.name = name
        self.age = age

# Create object
s1 = Student("Ehsan", 20)
print(s1.name)  # Ehsan
print(s1.age)   # 20
```

When we do `Student("Ehsan", 20)`, Python automatically calls:

- Student.__init__(s1, "Ehsan", 20)

What is self?

- `self` represents the **current object**.
- It allows each object to **store its own data**.
- Without `self`, Python wouldn't know which object's data you want to use.

```
class Student:
    def __init__(self, name, age):
        # this code runs automatically when a new Student is
created
        self.name = name
        self.age = age

# Create object
s1 = Student("Ehsan", 20)
S2 = Student("Nahid", 22)
print(s1.name)  # Ehsan
print(s1.age)   # 20

print(s2.name)  # Nahid
print(s2.age)   # 22
```

- Here, s1 and s2 each have their **own `name` and `age`** because of `self`.

Analogy

- **Class** → blueprint of a house.
- **Object** → a real house built from the blueprint.
- **__init__** → the constructor who sets up each house with specific paint color, size, etc.
- **self** → says *“this house”*, not some other house.

So:

- self.name = name → "give *this house* a name".
- self.age = age → "set the age for *this student*".

Summary:

- `__init__`: special method that **initializes an object’s data**.
- `self`: keyword that **represents the current object** (like *“this”* in Java, C++).

Class variable

- Belongs to the **class** (shared by all objects).
- Defined **outside methods** (but inside the class).
- Changing it affects all instances (unless shadowed by instance variable)

```
class Student:
    # Class variable
    school_name = "ABC High School"

    def __init__(self, name):
        self.name = name  # Instance variable

# Access class variable
s1 = Student("Ehsan")
s2 = Student("Hasan")

print(s1.school_name)  # ABC High School
print(s2.school_name)  # ABC High School

# Changing class variable
Student.school_name = "XYZ Academy"
print(s1.school_name)  # XYZ Academy
print(s2.school_name)  # XYZ Academy
```

Instance Variable

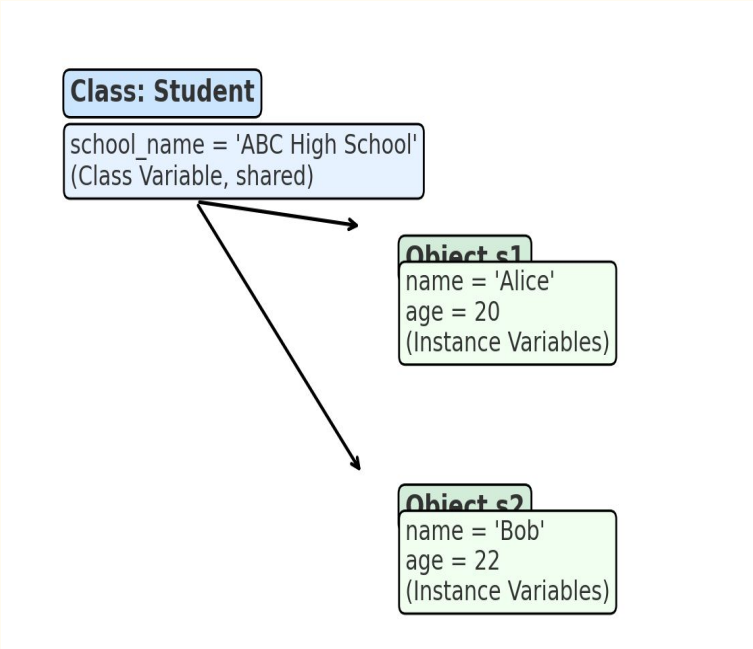
- Belongs to a **specific object** (different for each instance).
- Usually defined inside __init__() using self.
- Changing one object's instance variable **doesn't affect others**.

```
class Student:
    school_name = "ABC High School"  # Class variable

    def __init__(self, name, age):
        self.name = name  # Instance variable
        self.age = age    # Instance variable

# Creating objects
s1 = Student("Ehsan", 20)
s2 = Student("Nahid", 22)

print(s1.name, s1.age)  # Ehsan 20
print(s2.name, s2.age)  # Nahid 22

# Changing instance variable
s1.age = 21
print(s1.age)  # 21
print(s2.age)  # 22 (unchanged)
```

Key Differences

Feature	Class Variable	Instance Variable
Belongs to	Class	Individual object
Declared inside	Class (outside methods). But inside class	Inside `__init__`
Shared across objects	Yes	No
Example use	Common property (e.g., school name)	Unique property (e.g., student name, age)

Class Vs Instance Variables



Here's a **memory diagram** showing the difference:

- The **class variable (school_name)** lives inside the class, and both objects (s1, s2) point to it.
- The **instance variables (name, age)** live inside each object separately.

This way, changing s1.age won't affect s2.age, but changing Student.school_name updates it for all objects.

Thank you!